Tipos de Dados

Uma das tarefas mais importantes quando buscamos uma solução computacional para um problema é a escolha do *modelo abstrato* que representa os estados possíveis da realidade, desconsiderando aspectos que não são relevantes para o problema em questão. Este modelo descreve os *dados* sobre os quais os algoritmos irão atuar. A representação adequada da realidade normalmente envolve tipos de informação diferentes, como números, imagens, nomes, ou mesmo dados com estruturas internas complexas, portanto é necessário termos definições claras e precisas sobre os tipos de dados que serão utilizados na solução de cada problema.

Um *tipo de dados* é um conjunto de elementos que podem ser *atômicos* (sem estrutura interna, como Números, valores Booleanos, Símbolos, ...) ou *compostos* (com estrutura interna, como registros, listas, matrizes, árvores, grafos, ...). A maioria das linguagens de especificação/programação oferece vários tipos de dados básicos, normalmente chamados de *tipos pré-definidos*, bem como operações para trabalhar sobre estes tipos de dados, chamadas de *operações ou funções pré-definidas*. Além disso, as linguagens possuem também mecanismos para cada desenvolvedor poder criar novos tipos de dados (novos conjuntos) a partir de tipos já definidos. Normalmente, estes mecanismos são baseados em operações sobre conjuntos (produto cartesiano, união, ...), mas algumas linguagens também permitem a construção de tipos de dados através de definições recursivas.

A seguir, apresentaremos algumas formas de definir tipos de dados em Pyret. A seção 1 apresenta os tipos de dados pré-definidos da linguagem, bem como algumas operações disponíveis para estes tipos. Pode-se também construir tipos de dados por enumeração ou usando a união de tipos já definidos. Isso será mostrado na seção 2. A seção 3 mostra como construir tipos de dados utilizando-se a operação de produto cartesiano. Este tipo de dado, chamado de *estrutura* em Pyret, é muito utilizado para a construção de soluções computacionais. A seção 4 apresenta uma maneira de construir tipos de dados utilizando recursão. Os elementos destes conjuntos são dados estruturados, e a estrutura é definida por regras ou leis de formação. A construção de cenas a partir da inserção de várias imagens em uma cena vazia também pode ser construída usando uma definição recursiva (seção 5).

1 Tipos Pré-Definidos

A seguir, listamos alguns dos tipos pré-definidos em Pyret, bem como algumas operações que podem ser utilizadas para trabalhar com elementos destes tipos. Nas definições a seguir, o símbolo ___ será usado para representar o conjunto de todos os valores possíveis (ou seja, a união de todos os tipos).

1.1 Booleano

Booleano é o conjunto dos valores verdade verdadeiro (true) e falso (false), ou seja

$$\text{Booleano} = \{\text{true}, \text{false}\}$$

Algumas operações sobre Booleanos são:

not : Booleano → Booleano

Obj: Esta função tem como resultado a negação do valor verdade passado como argumento

Exemplo(s):
not(true) = false
not(false) = true

and : Booleano Booleano → Booleano

Obj: Esta função tem como resultado a conjunção dos valores Booleanos passados como argumento.

Exemplo(s):
true and true = true
true and false = false

or : Booleano Booleano → Booleano

Obj: Esta função tem como resultado a disjunção dos valores Booleanos passados como argumento.

Exemplo(s):
true or true = true
true or false = true

is-boolean : ___ → Booleano

Obj: Dado um argumento de qualquer tipo, esta função devolve verdadeiro se este argumento for um valor Booleano, caso contrário devolve falso.

Exemplo(s):
is-boolean(true) = true
is-boolean(20) = false

1.2 Número

Número é o conjunto dos Números. Números inexatos (aproximados) são representados com os caracteres `#i` no início (por exemplo, #i1.5 indica que este é um número é aproximadamente 1.5). Existem várias operações pré-definidas sobre números:

\+ : Número Número → Número

Obj: Esta função tem como resultado a soma dos dois números passados como argumento.

Exemplo(s):
2 + 5 = 7
60 + -2 = 58

\- : Número Número → Número

Obj: Esta função tem como resultado a subtração dos números passados como argumento.

Exemplo(s):
2 - 5 = -3
60 - -2 = 62

* : Número Número → Número

Obj: Esta função tem como resultado a multiplicação dos dois números passados como argumento.

Exemplo(s):
2 * 5 = 10
60 * -2 = -120

/ : Número Número → Número

Obj: Esta função tem como resultado a divisão dos números passados como argumento. Caso o segundo argumento seja zero, ocorre um erro de execução (divisão por zero).

Exemplo(s):
6 / 3 = 2
60 / -2 = -30

num-sqr : Número → Número

Obj: Calcula o quadrado do número passado como argumento.

Exemplo(s):
num-sqr(5) = 25
num-sqr(-2) = 4

num-sqrt : Número → Número

Obj: Calcula a raiz quadrada do número passado como argumento. O número deve ser positivo, senão ocorrerá um erro de execução.

Exemplo(s):
num-sqrt(25) = 5
num-sqrt(1.5) = ∼1.224744871391589 (a notação ∼ indica que este resultado é um número aproximado)

num-expt : Número Número → Número

Obj: Dados 2 números b e e, nesta ordem, calcula b^e.

Exemplo(s):
num-expt (6, 2) = 36
num-expt (4, -1) = 0.25
num-expt (4, 1/2) = 2

num-round : Número → Número

Obj: Arredonda o número passado como argumento para o inteiro mais próximo.

Exemplo(s):

```
num-round ( 5) = 5
num-round ( -1.2) = -1
num-round( 1.5) = 2
```

num-ceiling : Número → Número

Obj: Arredonda o número passado como argumento para o menor inteiro que é maior que o número.

Exemplo(s):

```
num-ceiling ( 5) = 5
num-ceiling ( -1.2) = -1
num-ceiling( 1.1) = 2
```

num-to-roughnum : Número → Número

Obj: Transforma um número exato em um inexato (aproximado). O padrão em Pyret*é considerar um número como exato, e por isso* 1.5 *normalmente é representado internamente como a fração* 3/2*, a dízima* 0.3333... *é representada internamente como* 1/3*. Mas pode-se forçar a representação decimal ao invés da fracionária usando esta função. Note que pode haver perda de exatidão nesta transformação (por exemplo, no caso do número* 1/3*).*

Exemplo(s):

```
num-to-roughnum( 5) = ~5.0
num-to-roughnum( 1 / 3) = ~0.3333333333333333
```

num-to-string : Número → String

Obj: Transforma um número em uma string (palavra). Quando números não inteiros em notação decimal são transformados, é usada a notação fracionária (divisão de dois inteiros).

Exemplo(s):

```
num-to-string( 5) = "5"
num-to-string ( -1) = "-1"
num-to-string( 1.5) = "3/2"
num-to-string( num-to-roughnum(1.5)) = "~1.5"
```

num-modulo : Número Número → Número

Obj: Dados 2 números a e b, nesta ordem, devolve o resto da divisão inteira de a por b.

Exemplo(s):

```
num-modulo( 6, 2) = 0
num-modulo( 7, 2) = 1
num-modulo( 234, 12) = 6
```

num-truncate : Número → Número

Obj: Devolve a parte inteira do número.

Exemplo(s):

```
num-truncate( 3.14) = 3
num-truncate( -3.14) = -3
```

num-sin : Número → Número

Obj: Dado um numero representando um ângulo (em radianos) devolve o seno deste número. O resultado normalmente é um número inexato. Existem também as funções **`num-cos`** *e* **`num-tan`***, para calcular cosseno e tangente, respectivamente.*

Exemplo(s):

```
num-sin( 0) = 0
num-sin( 3.141592) = ~6.535897930762419e-7
num-sin( (/ 3.141592 2)) = ~0.9999999999999466
num-sin( (/ PI 2)) = ~1
```

num-random : Número → Número

Obj: Dado um numero positivo diferente de zero k devolve um número inteiro aleatório entre 0 e $k-1$.

Exemplo(s):

```
num-random( 1) = 0
num-random( 20) = 18
```

1.3 String

Strings são sequências de caracteres do teclado. Em Pyret, são representados entre aspas:

String ={ "0", "casa", "A", "Erro", "Bom Dia!", ...}

Algumas operações que se pode realizar sobre Strings são:

is-string : ___ → Booleano

Obj: Dado um argumento de qualquer tipo, esta função devolve verdadeiro se este argumento for um valor do tipo String*, caso contrário devolve falso.*

Exemplo(s):

is-string("casa") = true
is-string(20) = false
is-string("20") = true

string-equal : String String → Booleano

Obj: Dados dois argumentos do tipo String*, devolve verdadeiro se eles forem iguais, caso contrário devolve falso.*

Exemplo(s):

string-equal("casa" "casa") = true
string-equal("20" "2 0") = false

string-length : String → Número

Obj: Dado um argumento do tipo String*, devolve o número de caracteres da string.*

Exemplo(s):

string-length ("casa") = 4
string-length("20 2 0!! z") = 10

string-append : String String → String

Obj: Dados dois argumentos do tipo String*, constrói uma string unindo os argumentos.*

Exemplo(s):

string-append ("casa" , "casa") = "casacasa"
string-append("20 ", "casas") = "20 casas"

1.4 Imagem

Imagem é um tipo de dados usado em Pyret para tratar imagens (em vários formatos). As imagens são consideradas ítens indivisíveis, como os símbolos. As operações sobre imagens são análogas às de símbolos, is-image para testar se um elemento é uma imagem, e images-equal para verificar se duas imagens são iguais.

is-image : ___ → Booleano

Obj: Dado um argumento de qualquer tipo, esta função devolve verdadeiro se este argumento for um valor do tipo Imagem*, caso contrário devolve falso.*

Exemplo(s):

is-image ("casa") = false
is-image(20) = false
is-image( ) = true

images-equal : Imagem Imagem → Booleano

Obj: Dados dois argumentos do tipo Imagem*, devolve verdadeiro se eles forem iguais, caso contrário devolve falso.*

Exemplo(s):

images-equal(,) = true

images-equal(,) = false

text : String Number String → Image

Obj: text *é uma função que recebe uma palavra (string), um número, que é o tamanho do fonte a ser utilizado, e a cor na qual o texto deve ser escrito (string). O resultado é uma imagem.*

Exemplo(s):

text("Hello", 20, "green") = Hello

circle : Number String String → Image

Obj: Dados o raio do círculo, o tipo de preenchimento e a cor, desenha a imagem do círculo correspondente.

Exemplo(s):

circle(20, "solid", "red") =

star : Number String String → Image

Obj: Dados a distância entre as pontas, o tipo de preenchimento e a cor, desenha a imagem da estrela correspondente.

Exemplo(s):

star(20, "solid", "yellow") =

rectangle : Number Number String String → Image

Obj: Dados os tamanhos dos lados, o tipo de preenchimento e a cor, desenha a imagem do retângulo correspondente.

Exemplo(s):

rectangle(40, 20, "outline", "skyBlue") =

ellipse : Number Number String String → Image

Obj: Dados a largura, a altura, o tipo de preenchimento e a cor, desenha a imagem da elipse correspondente.

Exemplo(s):

elipse(40, 20, "solid", "darkGreen") =

overlay : Image Image → Image

Obj: Dadas duas imagens, gera uma imagem com as imagens sobrepostas, a primeira ficará em cima. As imagens são alinhadas pelos seus pinholes.

Exemplo(s):

overlay( ) =

overlay-xy : Image Image → Image

Obj: Dados uma imagem, as coordenadas x e y do descolamento e outra imagem, gera uma imagem com a primeira imagem sobreposta à segunda, levando em consideração o descolamento da segunda imagem em x unidades para a direita e y para baixo. Inicialmente, as imagens estão alinhadas pelos seus cantos esquerdos superiores.

Exemplo(s):

overlay-xy(, 10, 5,) =

above : Image Image → Image

Obj: Dadas duas imagens, gera uma imagem com as imagens uma acima da outra, a primeira ficará no topo.

Exemplo(s):

above(,) =

above-align : String Image Image → Image

Obj: Dadas uma palavra, que pode ser "left", "right" ou "middle" e duas imagens, gera uma imagem com as imagens uma acima da outra, alinhando horizontalmente de acordo com a palavra.

Exemplo(s):

above-align("right",) = ,

beside : Image Image → Image

Obj: Dadas duas imagens, gera uma imagem com as imagens uma ao lado da outra, a primeira ficará à esquerda.

Exemplo(s):

beside(,) =

beside-align : String Image Image → Image

Obj: Dadas uma palavra, que pode ser "top", "bottom", "middle" ou "baseline" e duas imagens, gera uma imagem com as imagens uma ao lado da outra alinhando verticalmente de acordo com a palavra

Exemplo(s):

beside/align("top", ,) =

flip-horizontal : Image → Image

Obj: Dada uma imagem, gira a imagem no seu eixo horizontal.

Exemplo(s):

flip-horizontal( ) =

rotate : Image Number → Image

Obj: Dados um ângulo (em graus) e uma imagem, gira a imagem de acordo com o ângulo.

Exemplo(s):

rotate(30,) =

2 Enumerações e Tipos Mistos

Em várias situações precisamos definir conjuntos finitos para serem usados como entrada ou saída de funções. Por exemplo:

- o conjunto das letras do alfabeto;
- o conjunto dos dígitos de 0 a 9;
- o conjuntos de bases nitrogenadas do DNA;
- o conjunto de meses do ano;
- o conjunto com mensagem(ou mensagens) de erro

Nestes casos, podemos *enumerar* os elementos dos conjuntos para defini-los:

```
;; Letras = {"A","B","C","D","E","F","G","H","I","J","K","L","M","N","O","P","Q","R","S"."T","U","V","W","X","Y","Z"}
;; Dígitos = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
;; Bases = {"A", "T", "C", "G"}
;; Meses = {"jan", "fev", "mar", "abr", "mai", "jun", "jul", "ago","set", "out", "nov", "dez"}
;; Erro = {"Erro"}
```

Muitas vezes o resultado do processamento de uma função pode ser de um tipo ou outro, dependendo dos argumentos passados para ela. Isto pode ocorrer, por exemplo nos casos em que uma função realiza algum cálculo que deve resultar em um número, caso os argumentos sejam positivos, ou devolve uma mensagem de erro, caso eles sejam negativos. Para estes casos, bem como em várias outras situações, é útil definirmos um tipo de dados que representa

a união de conjuntos. Como os elementos do conjunto resultante podem ser de um tipo ou outro, estes tipos são chamados *tipos mistos*.

Exemplos:

```
;; Um NúmeroOUerro é
;;     1. um elemento de Número, ou
;;     2. um elemento de Erro
```

```
;; Um LetraOUdígito é
;;     1. um elemento de Letras, ou
;;     2. um elemento de Dígitos
```

Em notação matemática, estes conjuntos podem ser definidos como:

$$\text{NúmeroOUerro} = \text{Número} \cup \text{Erro}$$

$$\text{LetraOUdígito} = \text{Letras} \cup \text{Dígitos}$$

3 Tipos Compostos: Estruturas

Em muitas situações, é interessante guardarmos informações relacionadas em um único bloco de dados. Por exemplo, gostaríamos de definir os seguintes tipos de dados (conjuntos):

Posn : conjunto de coordenadas de pontos no plano. Um ponto é composto por 2 informações: o valor no eixo x e o valor no eixo y;

Notas : conjunto das notas das provas 1 e 2. Cada item deste conjunto contém 2 informações: a nota da primeira e a nota da segunda prova de um aluno;

ItemCadastro : conjunto de ítens de um cadastro. Pode-se imaginar um cadastro no qual constam nome, endereço e telefone de pessoas. Cada unidade deste cadastro é então composta por estas 3 informações;

Circulo : conjunto dos círculos, onde cada círculo é definido por 3 informações: seu ponto central, seu raio e sua cor;

Livro : conjunto dos livros, onde cada livro é definido por seu nome, autor, ano de publicação e preço.

Em cada um destes tipos listados acima, os elementos não são dados atômicos, mas tuplas (pares, triplas, quádruplas). Em Pyret a maneira de definir estes tipos de dados é utilizando a construção de *estruturas*, que é fundamentalmente a operação de produto cartesiano de conjuntos.

Por exemplo, poderiamos definir Posn e Notas como

$$\text{Posn} = \text{Número} \times \text{Número}$$
$$\text{Notas} = \text{Número} \times \text{Número}$$

Note que, tanto os elementos do conjunto Posn quanto os do conjunto Notas são pares ordenados de números. Para diferenciar os elementos destes conjuntos, pode-se indexar ou marcar cada elemento com o nome do conjunto (por exemplo, em vez de escrever $(1, 2)$ pode-se usar $(1, 2)_{\text{Posn}}$ ou (make-posn 1 2)).

Em Pyret, a definição de um conjunto de tuplas se dá através do comando define-struct. O conjunto Posn pode ser definido como:

```
(define-struct posn (x y))
;; Um elemento posn de Posn é uma estrutura
;; (make-posn  coord-x  coord-y), onde
;;              coord-x :  Número é a coordenada no eixo x do ponto
;;              coord-y :  Número é a coordenada no eixo y do ponto
```

Em notação matemática, este conjunto pode ser descrito como:

$$\text{Posn} = \{(\text{make-posn}\, coord-x\, coord-y) | coord-x \in \text{Número}\ e\ coord-y \in \text{Número}\}$$

Ou seja,

$$\text{Posn} = \{(\text{make-posn } 0\ 0), (\text{make-posn } 1\ 2), (\text{make-posn } -1\ 5), \ldots\}$$

Na definição de um elemento posn, x e y são chamados de atributos ou campos de um posn.
Outros exemplos:

```
(define-struct notas (prova1 prova2))
;; Um elemento notas de Notas é uma estrutura
;; (make-notas x y), onde
;;                    x : Número é a nota da prova 1
;;                    y : Número é a nota da prova 2
```

```
(define-struct círculo (centro raio cor))
;; Um elemento círculo de Círculo é uma estrutura
;; (make-círculo um-centro um-raio uma-cor), onde
;;                    um-centro : Posn é o ponto central do círculo
;;                    um-raio : Número é o raio do círculo
;;                    uma-cor : Símbolo representa a cor do círculo
```

A regra geral da definição de uma estrutura nomeEstrutura ∈ NomeEstrutura em Pyret é conforme abaixo, onde os conjuntos Tipo1 a TipoN podem ser quaisquer tipos já definidos:

```
(define-struct nomeEstrutura (campo1 campo2 ... campoN))
;; Um elemento nomeEstrutura de NomeEstrutura é uma estrutura
;; (make-nomeEstrutura x1 x2 ... xN), onde
;;                    x1 : Tipo1 é ...
;;                    x2 : Tipo2 é ...
...
;;                    xN : TipoN é ...
```

Quando se define uma estrutura nomeEstrutura ∈ NomeEstrutura em Pyret, automaticamente são geradas funções para trabalhar com esta estrutura. Estas funções são:

make-nomeEstrutura : Tipo1 Tipo2 ... TipoN → NomeEstrutura

Obj: Esta função gera elementos do tipo NomeEstrutura.

Exemplo(s):
make-notas(9 10) = gera um elemento do conjunto Notas (de acordo com a estrutura definida acima).

nomeEstrutura-campo1 : NomeEstrutura → Tipo1

Obj: Esta função tem como resultado o valor do primeiro campo do elemento de NomeEstrutura *passado como argumento.*

Exemplo(s):
notas-prova1 ((make-notas 9 10)) = 9

nomeEstrutura-campoN : NomeEstrutura → TipoN

Obj: Esta função tem como resultado o valor do N-ésimo campo do elemento de NomeEstrutura *passado como argumento.*

Exemplo(s):
notas-prova2 ((make-notas 9 10)) = 10

nomeEstrutura? : ___ → Booleano

Obj: Dado um argumento de qualquer tipo, esta função devolve verdadeiro se este argumento for um valor do tipo NomeEstrutura*, caso contrário devolve falso.*

Exemplo(s):
notas?((make-notas 9 10)) = true
notas?(20) = false
notas?((make-posn 0 0)) = false

4 Tipos Compostos: Listas

Listas é um tipo de dados muito utilizado em Computação. Pode ser utilizado sempre que tivermos um número arbitrário (mas finito) de elementos de determinados tipos que queremos armazenar de forma conjunta – por exemplo, notas dos alunos de uma turma, itens de um cadastro, cadeias de DNA (listas de bases nitrogenadas), descendentes de uma pessoa, ...

A definição de listas é recursiva, ou seja, utiliza-se a noção de lista na sua própria definição. Para uma definição recursiva ser bem formada, é necessário que se utilize (no mínimo) uma base, que no caso das listas é a lista vazia, ou empty, e um operador para construir uma lista, dada outra lista. No caso do Pyret, usamos a função cons para isso, que, dado um elemento e uma lista, constrói uma lista colocando este elemento na frente da lista dada. Uma lista de números pode ser definida por:

```
;; Uma Lista-de-números é
;;      - empty, ou
;;      - (cons  n ldn), onde
;;              n :  Número
;;              ldn : Lista-de-números
```

Esta definição cria o conjunto Lista-de-números, que contém todas as listas finitas cujos elementos são números, ou seja:

```
Lista-de-números = {empty,
                    (cons 0 empty),
                    (cons 2 (cons 0 empty)),
                    (cons -1 (cons 0 (cons 3 empty))),
                    ...}
```

Pode-se, também, definir conjuntos de listas com tamanho fixo, por exemplo, o conjunto de todas as listas de 2 números (que poderia representar o conjunto de pontos no plano):

```
;; Um elemento do conjunto Pontos tem o formato
;;      (cons  x (cons  y empty), onde
;;              x :  Número
;;              y:  Número
```

Para se trabalhar com listas, existem as seguintes funções pré-definidas (entre outras):

cons : TipoElem TipoLista → TipoLista

Obj: Dados um elemento e uma lista de elementos, gera uma nova lista com o elemento dado na frente.

Exemplo(s):

cons(2 empty) = (cons 2 empty) ⇒ *é a lista contendo o elemento 2*
cons(2 (cons 3 empty)) = (cons 2 (cons 3 empty)) ⇒ *é a lista contendo os elementos 2 e 3, nesta ordem*
cons(2 (cons 2 (cons 3 empty))) = (cons 2 (cons 2 (cons 3 empty)))⇒ *é a lista contendo os elementos 2, 2 e 3.*

first : TipoLista → TipoElem

Obj: Dada uma lista de elementos do tipo TipoElem*, esta função devolve seu primeiro elemento.*

Exemplo(s):

first ((cons 2 empty)) = 2
first(empty) = gera um erro de execução
first (2) = gera um erro de execução [argumento não é uma lista]

rest : TipoLista → TipoLista

Obj: Dada uma lista, esta função retira seu primeiro elemento e devolve a lista com o restante dos elementos.

Exemplo(s):

rest((cons 2 empty)) = empty
rest (empty) = gera um erro de execução

empty? : TipoLista → Booleano

Obj: Dada uma lista, esta função devolve verdadeiro se a lista for vazia, caso contrário devolve falso.

Exemplo(s):

empty?((cons 2 empty)) = false
empty?(empty) = true

cons? : TipoLista → Booleano

Obj: Dada uma lista, esta função devolve verdadeiro se a lista for não vazia, caso contrário devolve falso.

Exemplo(s):

cons?((cons 2 empty)) = true
cons?(empty) = false

append : TipoLista ... TipoLista → TipoLista

Obj: Dadas duas (ou mais) listas, esta função junta todas as listas em uma só, na ordem na qual foram passadas como argumento.

Exemplo(s):

append((cons 2 empty) (cons 2 empty)) = (cons 2 (cons 2 empty))
append(2 (cons 2 empty)) = gera um erro de execução [segundo argumento não é uma lista]

Outros exemplos de conjuntos de listas:

- Conjunto de listas de strings:

```
;; Uma Lista-de-strings é
;;      - empty, ou
;;      - (cons  s lds), onde
;;              s :  String
;;              lds : Lista-de-strings
```

- Conjunto de cadeia de DNA:

```
;; Uma Cadeia-de-DNA é
;;      - empty, ou
;;      - (cons  b cadeia), onde
;;              b : Bases
;;              cadeia : Cadeia-de-DNA
```

- Conjunto de cadeias de DNA com brancos:

```
;; Uma Cadeia-de-DNA-com-brancos é
;;      - empty, ou
;;      - (cons  b cadeia), onde
;;              b : Bases
;;              cadeia : Cadeia-de-DNA-com-brancos, ou
;;      - (cons  b cadeia), onde
;;              b : {"_"}
;;              cadeia : Cadeia-de-DNA-com-brancos
```

- Conjunto de listas de números e strings:

```
;; Uma Lista-numeros-strings é
;;      - empty, ou
;;      - (cons  n lns), onde
;;              n :  Número
;;              lns : Lista-numeros-strings, ou
;;      - (cons  s lns), onde
;;              s :  String
;;              lns : Lista-numeros-strings
```

- Conjunto de vetores de 3 posições contendo círculos (o conjunto Círculo foi definido na seção 3):

```
;; Um elemento do conjunto Vetor3Círculos tem o formato
;;        (cons  c1 (cons  c2 (cons  c3  empty))), onde
;;                c1,c2,c3: Círculo
```

- Conjunto de matrizes 5x5 de números:

```
;; Um elemento do conjunto Linha tem o formato
;;        (cons  n1 (cons  n2 (cons  n3 (cons  n4 (cons  n5 empty))))), onde
;;                n1,n2,n3,n4,n5:  Número

;; Um elemento do conjunto Matriz tem o formato
;;        (cons  n1 (cons  n2 (cons  n3 (cons  n4 (cons  n5 empty))))), onde
;;                n1,n2,n3,n4,n5: Linha
```

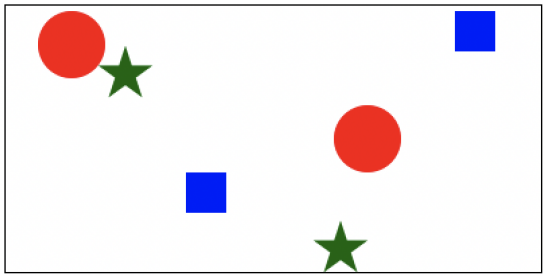
Ao invés dos construtores cons e empty, pode-se usar a abreviação list:

(cons *n1* (cons *n2* (cons *n3* (cons *n4* (cons *n5* empty))))) ≡ (list *n1 n2 n3 n4 n5*)

5 Tipos Compostos: Cenas

A linguagem Pyret provê um tipo de dados para representar uma cena. Uma cena é composta pos várias imagens, colocadas sobre uma tela vazia. Podemos usar uma definição recursiva para construir cenas:

```
;; Uma Cena é
;;        - (empty-scene  larg  alt), onde
;;                larg :  Número, é a largura da cena
;;                alt:  Número, é a altura da cena, ou
;;        - (place-image  img x y c), onde
;;                img :  Imagem, é a imagem que será colocada na cena,
;;                x , y:  Número, são as coordenadas (x,y) onde a imagem será colocada,
;;                c : Cena, é a cena onde a imagem será colocada.
```

Por exemplo, a imagem a seguir à direita é gerada pelo código à esquerda:

```
(define CIRCULO-VERMELHO (circle 25 "solidred"))
(define QUADRADO-AZUL (rectangle 30 30 "solidblue"))
(define ESTRELA-VERDE (star 25 "soliddarkgreen"))

(place-image QUADRADO-AZUL 350 20
  (place-image CIRCULO-VERMELHO 270 100
    (place-image ESTRELA-VERDE 90 50
      (place-image ESTRELA-VERDE 250 180
        (place-image QUADRADO-AZUL 150 140
          (place-image CIRCULO-VERMELHO 50 30
            (empty-scene 400 200)))))))
```